FIREBASE FIRESTORE CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. OVERVIEW & SETUP

Installation

Firebase Firestore powerful database solution

```
# Install (Docker):
docker run -d --name mydb \
-e DB_PASSWORD=secret \
-p 5432:5432 firebase firestore:latest
```

Connect: host, port, user, password, database name

02. INSTALLATION & CONFIG

Architecture

```
# Package manager install
brew install firebase firestore
# Or download from official site
```

Config file: firebase firestore.conf or similar
Set max connections, memory, storage paths

```
# Start service
systemctl start firebase firestore
```

03. DATA MODELING

YAML / Config

Define schemas, tables/collections, data types

```
-- Create table (SQL example):
CREATE TABLE users (
id SERIAL PRIMARY KEY,
name VARCHAR(100) NOT NULL,
email VARCHAR(200) UNIQUE NOT NULL
);
```

Normalize for consistency; denormalize for performance

04. CRUD OPERATIONS

Workflow

```
-- INSERT
INSERT INTO users (name, email) VALUES ('Alic...
-- SELECT
SELECT * FROM users WHERE active = true;
-- UPDATE
UPDATE users SET name = 'Bob' WHERE id = 1;
-- DELETE
DELETE FROM users WHERE id = 1;
```

05. QUERYING

CRUD API

```
SELECT u.*, o.total
FROM users u
JOIN orders o ON o.user_id = u.id
WHERE o.status = 'paid'
ORDER BY o.created_at DESC
LIMIT 10 OFFSET 20;
```

Use EXPLAIN/EXPLAIN ANALYZE to inspect query plans

06. INDEXING

Integration

```
CREATE INDEX idx_users_email ON users(email);
CREATE UNIQUE INDEX idx_users_email_unique ON...
-- Composite index:
CREATE INDEX idx_orders ON orders(user_id, st...
```

Index columns used in WHERE, JOIN, ORDER BY clauses
Too many indexes slow writes profile before adding

SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. TRANSACTIONS

Hardening

```
BEGIN;
UPDATE accounts SET balance = balance - 100 W...
UPDATE accounts SET balance = balance + 100 W...
COMMIT;
-- On error:
ROLLBACK;
```

ACID: Atomicity, Consistency, Isolation, Durability

08. REPLICATION & HA

Observability

Primary-replica replication for read scaling

```
# Primary: enable binary logging / WAL
# Replica: point to primary host
```

Automatic failover with Patroni/Sentinel/etc
Read from replicas; write to primary only
Monitor replication lag alert if > threshold

09. SECURITY

Optimization

Create least-privilege database users

```
CREATE USER appuser WITH PASSWORD 'secret';
GRANT SELECT, INSERT, UPDATE ON users TO appu...
```

Enable SSL/TLS for connections in transit
Encrypt sensitive columns at application layer
Regular security updates and patches

10. MONITORING & PERF

Unit Tests

Monitor query performance, connections, disk I/O

```
-- Slow query log
SET log_min_duration_statement = 1000;  -- 1s
```

EXPLAIN ANALYZE for individual query tuning
Connection pooling: PgBouncer, ProxySQL, HikariCP
Vacuum/ANALYZE (PostgreSQL) or OPTIMIZE (MySQL)

11. CLI / TOOLS

Commands

```
psql -h host -U user -d database
\l   list databases  \dt  list tables
\d users  describe table structure
-- Backup:
pg_dump dbname > backup.sql
-- Restore:
psql dbname < backup.sql
```

12. COMMON GOTCHAS

Warning

Never store plaintext passwords use bcrypt/Argon2
Use connection pooling raw connections are expensive
Always parameterize queries prevent SQL injection
Regular backups with point-in-time recovery
Test failover procedures before production incidents